

ARCHITECTURE REVIEW PRESENTATION

Hostel Buddy

Smart Hostel Complaint Management System

Group 19

Pranjal Tyagi · Ajinkya · Dinesh Saini · Saurabh · Vijay

Department of Computer Science and Engineering · **IIIT Kottayam**



Agenda & Speaker Allocation

20 minutes

#	Topic	Time	Presenter
1	Project Overview	1 min	Pranjal Tyagi
2	Module Decomposition	2 min	
3	Activity Diagram	1.5 min	
4	System Architecture	2.5 min	Ajinkya
5	Database Design — ER Diagram	1.5 min	
6	Class Diagram	1.5 min	Dinesh Saini
7	Layered Architecture & Data Flow	2.5 min	
8	Sequence Diagram	1.5 min	Saurabh
9	Security & Key Design Decisions	1.5 min	
10	State Diagram	1.5 min	Vijay
11	System Requirements & Use Case Diagram	2 min	
—	Conclusion	0.5 min	All

1 Project Overview

Pranjal Tyagi

1 min

Problem

- Hostel complaints are recorded in a **paper register**
- No way to search, track or report on them
- Students get **no visibility** after reporting an issue

Objective

- Digitise complaint reporting and resolution
- Give students **real-time status tracking**
- Give the administrator a **single dashboard** with analytics
- Store every complaint **permanently and searchably**

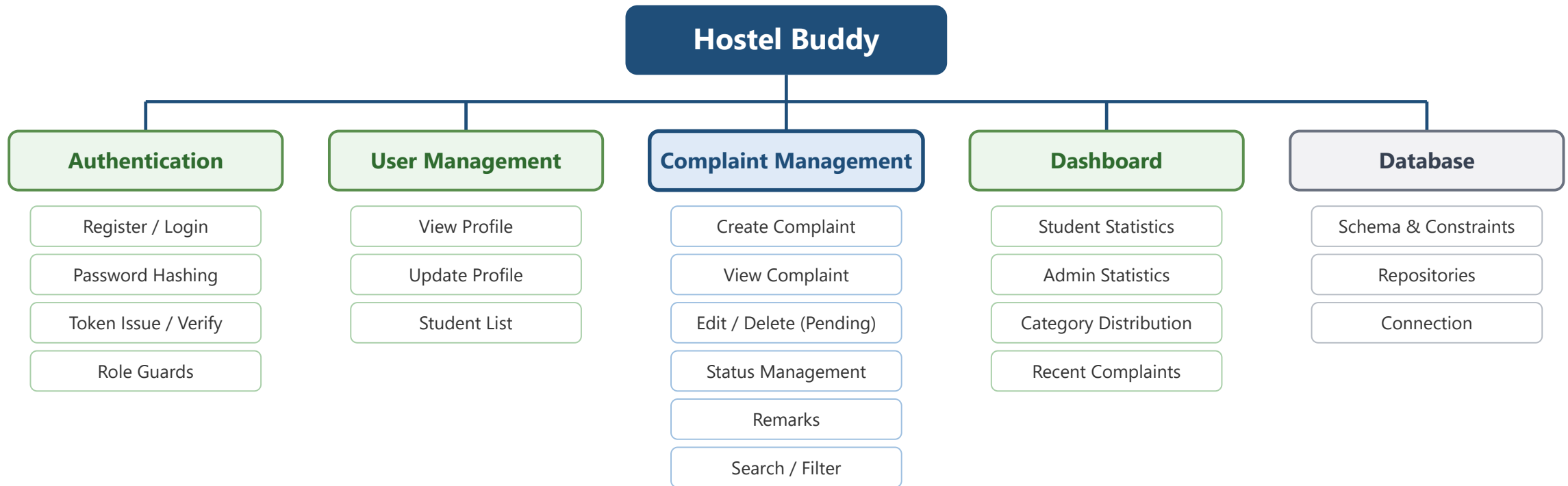
Who uses the system

- **Student** — raises and tracks their own complaints
- **Administrator** — reviews all complaints, updates status, views analytics

Two roles only — no separate maintenance-staff role in v1.0

How it works — in one line

A student submits a complaint through a web interface; it is stored as **Pending**, and the administrator advances it through **In Progress** → **Resolved** → **Closed** while the student watches the status change live.



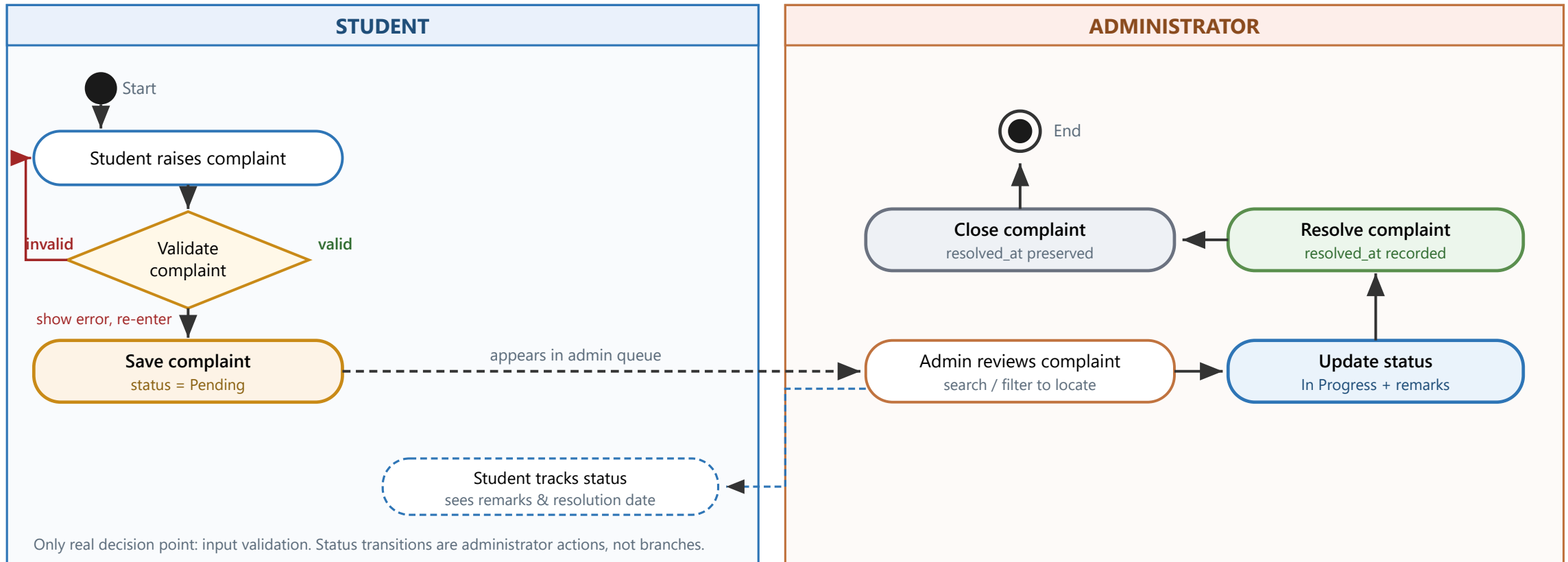
Purpose of decomposition

Each module has one specific responsibility — this makes the system easier to **maintain**, **test** and **extend**.

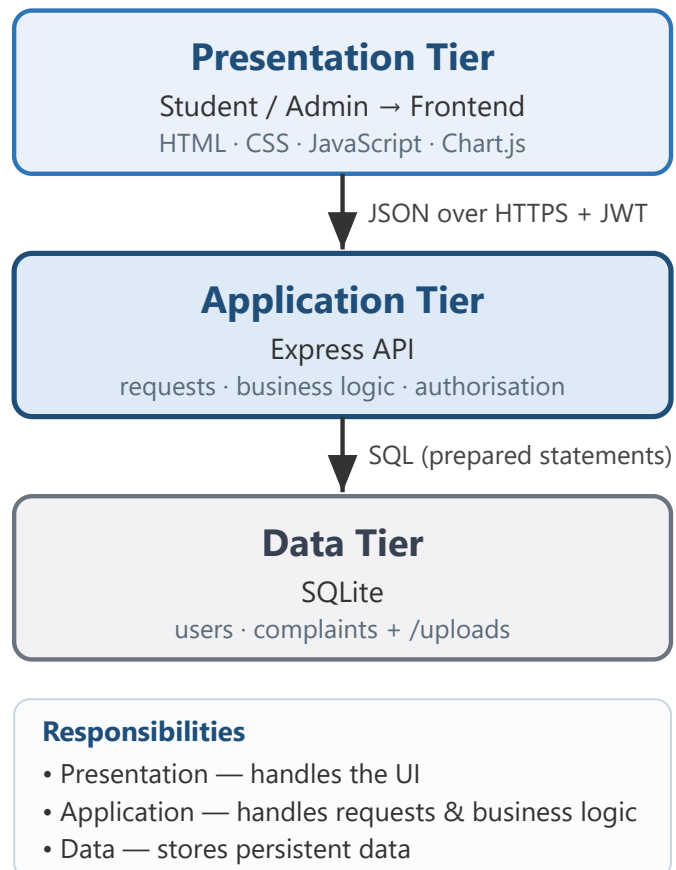
3 Activity Diagram — Complaint Process

Pranjal Tyagi

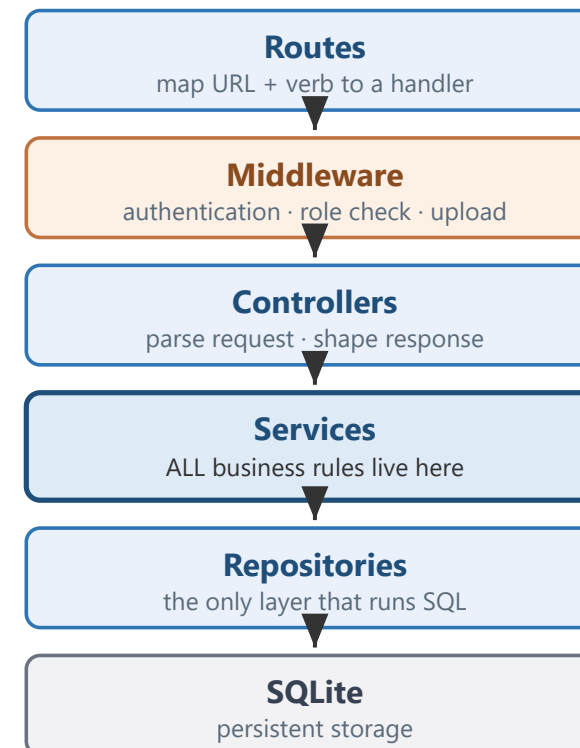
1.5 min



Three-Tier View



Inside the Application Tier

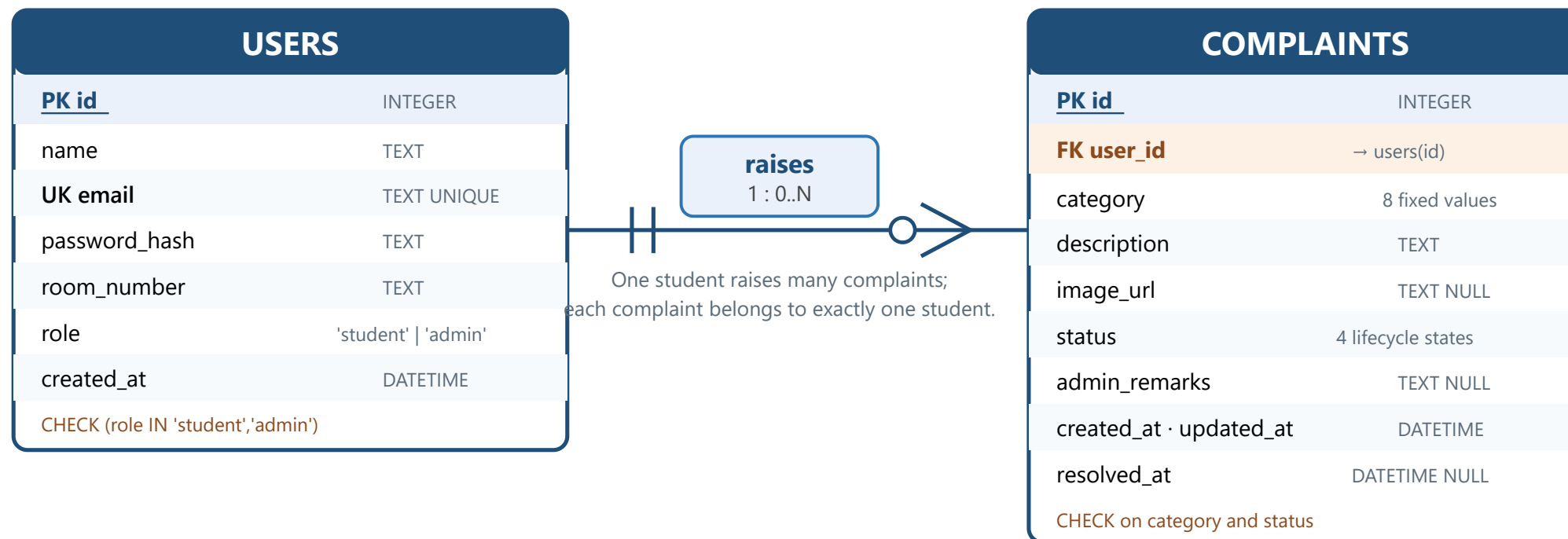


Each layer talks only to the layer directly below it.
Business rules never appear in the frontend or in SQL.

5 Database Design — ER Diagram

Ajinkya

1.5 min

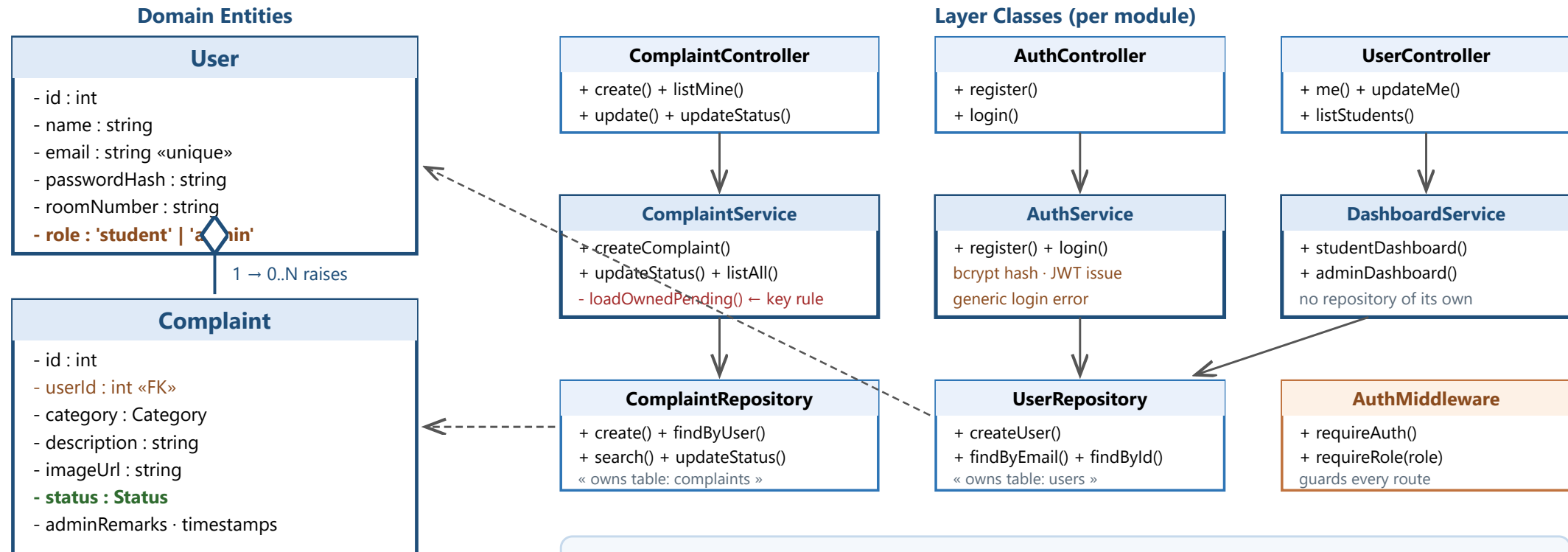


Two entities only. Admin is a **role** on USERS, not a separate table — authentication stays uniform. Categories and statuses are enforced by **CHECK constraints** rather than lookup tables — fixed sets, one fewer join.

6 Class Diagram

Dinesh Saini

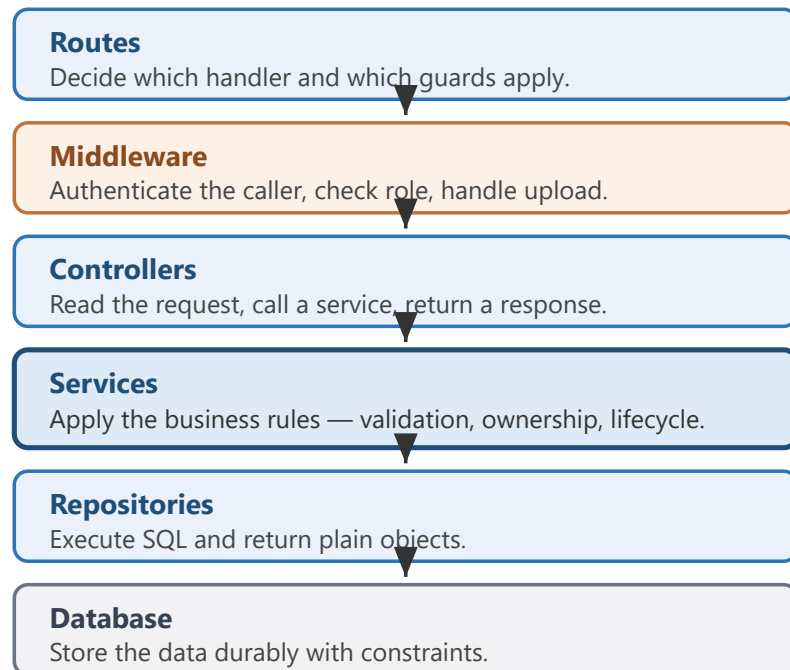
1.5 min



Reading the diagram

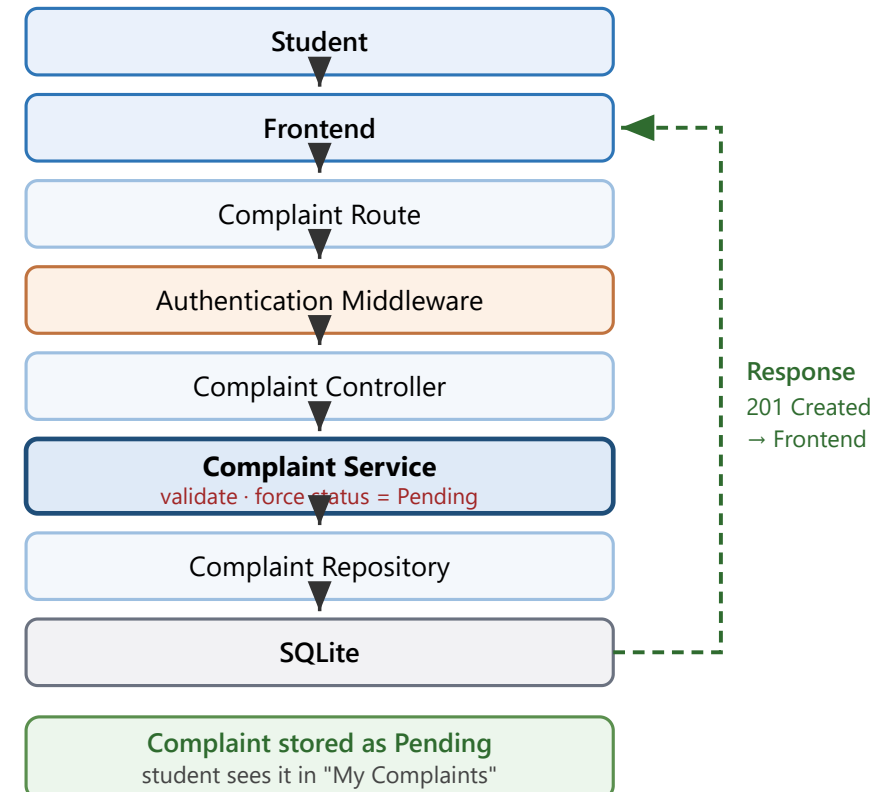
Controller → Service → Repository → Entity. Dependencies point downward only; no cycles.
Admin is **not a subclass** — it is the **role** attribute on User, so one class covers both actors.

Layer Responsibilities



A request can never skip a layer.

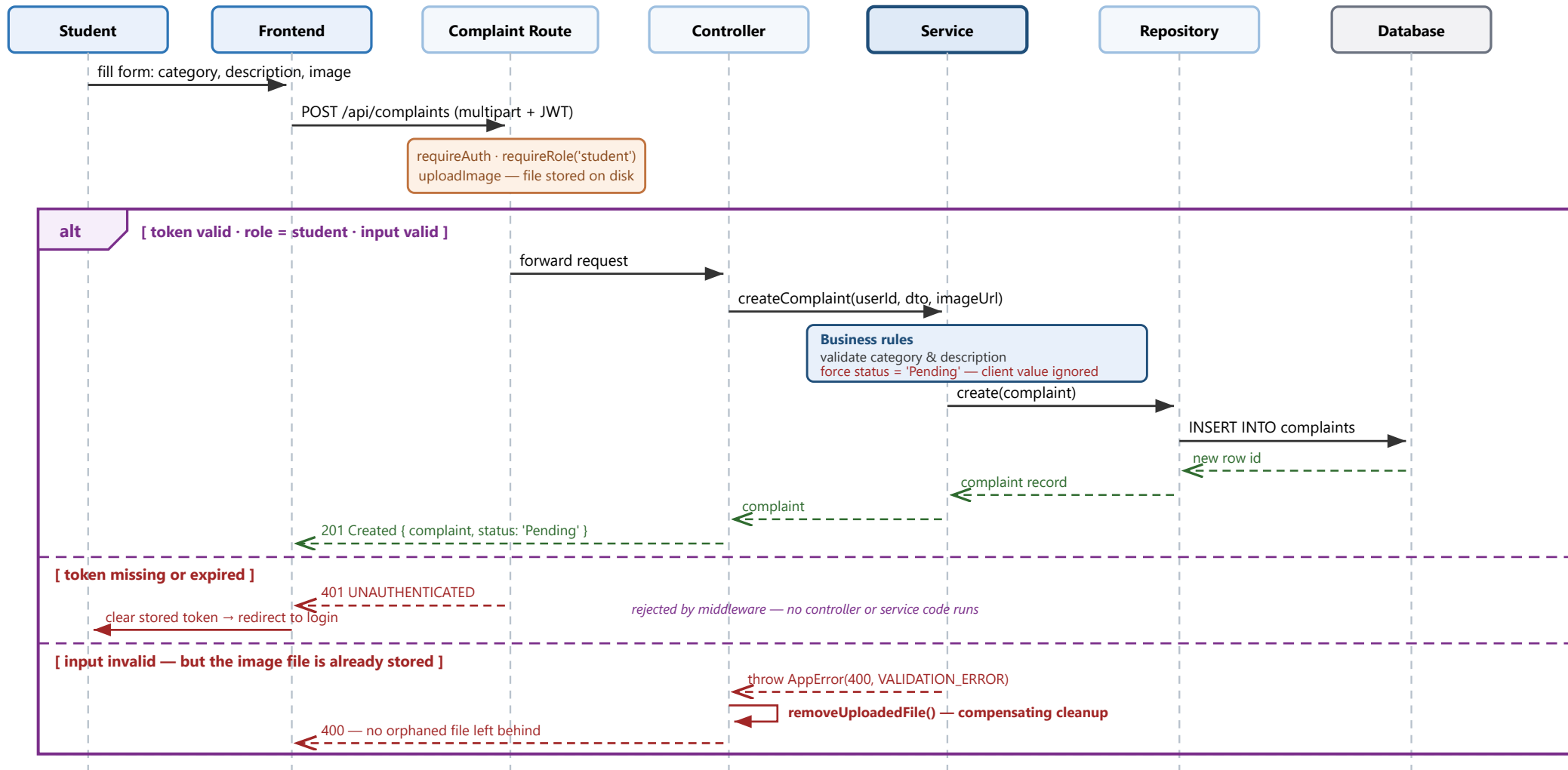
Concrete Request — Raise Complaint



8 Sequence Diagram — Raise Complaint, with Failure Paths

Saurabh

1.5 min



Fourth branch — PUT /complaints/:id where status ≠ Pending: the **Service** (not the Route) throws **409 NOT_PENDING** — a route-level check would protect only that one route, a service invariant holds for **every** caller.

Security controls

- **JWT authentication** — stateless signed tokens
- **bcrypt password hashing** — salted, never reversible
- **Role-based access control** — student vs administrator
- **Helmet / CSP** — headers; scripts restricted to our origin
- **Rate limiting** on authentication endpoints

Why SQLite?

Built into Node.js · no compiler or native build requirement · simple setup for the model version

Why layered architecture?

Separation of concerns · easier maintenance · easier testing · business logic independent from database access

Why a Repository layer?

Keeps SQL / database operations isolated · makes future database replacement easier

Accepted Trade-offs — limitations we chose knowingly

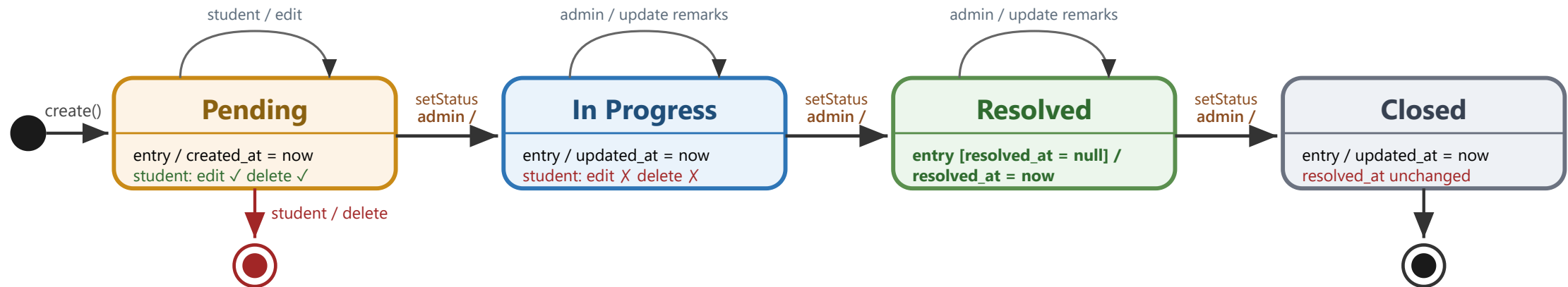
Every one of these is a deliberate decision with a bounded risk and a documented upgrade path.

Weakness we accepted	Why we accepted it	What bounds the risk
JWT cannot be revoked before it expires	Statelessness is what lets us replicate instances horizontally — a session store would reintroduce shared state	24-hour token lifetime caps the exposure window; a server-side deny-list or refresh-token scheme attaches at the existing verification middleware
Complaint images are served unauthenticated	Gating every image means streaming each one through an authorised route — real complexity for little gain at this scale	Filenames are timestamp + random hex, so they are impractical to guess, and URLs appear only inside access-controlled views
SQLite allows only a single writer	Complaint reads vastly outnumber writes; a separate database server adds a network hop and another failure mode	WAL mode preserves read concurrency; the PostgreSQL migration touches only the repository layer, never business logic

10 State Diagram — Complaint Lifecycle

Vijay

1.5 min



Lifecycle rules enforced by the system

- The student **creates** the complaint — it always starts in **Pending**; the **admin controls every transition**.
- **The student cannot change status** — there is no student operation that writes it.
- **resolved_at is recorded at Resolved**, once only — moving to Closed never overwrites it.
- Edit / delete exist **only in Pending**; a state outside these four is rejected by a database CHECK constraint.

Student Functional Requirements

- Register / Login
- Manage profile
- Raise complaint
- View complaints
- Edit / Delete **Pending** complaint
- Track complaint status
- View remarks and resolution date
- View personal statistics

Administrator Functional Requirements

- Login
- View all complaints
- Search / filter complaints
- Update complaint status
- Add remarks
- View registered students
- View analytics

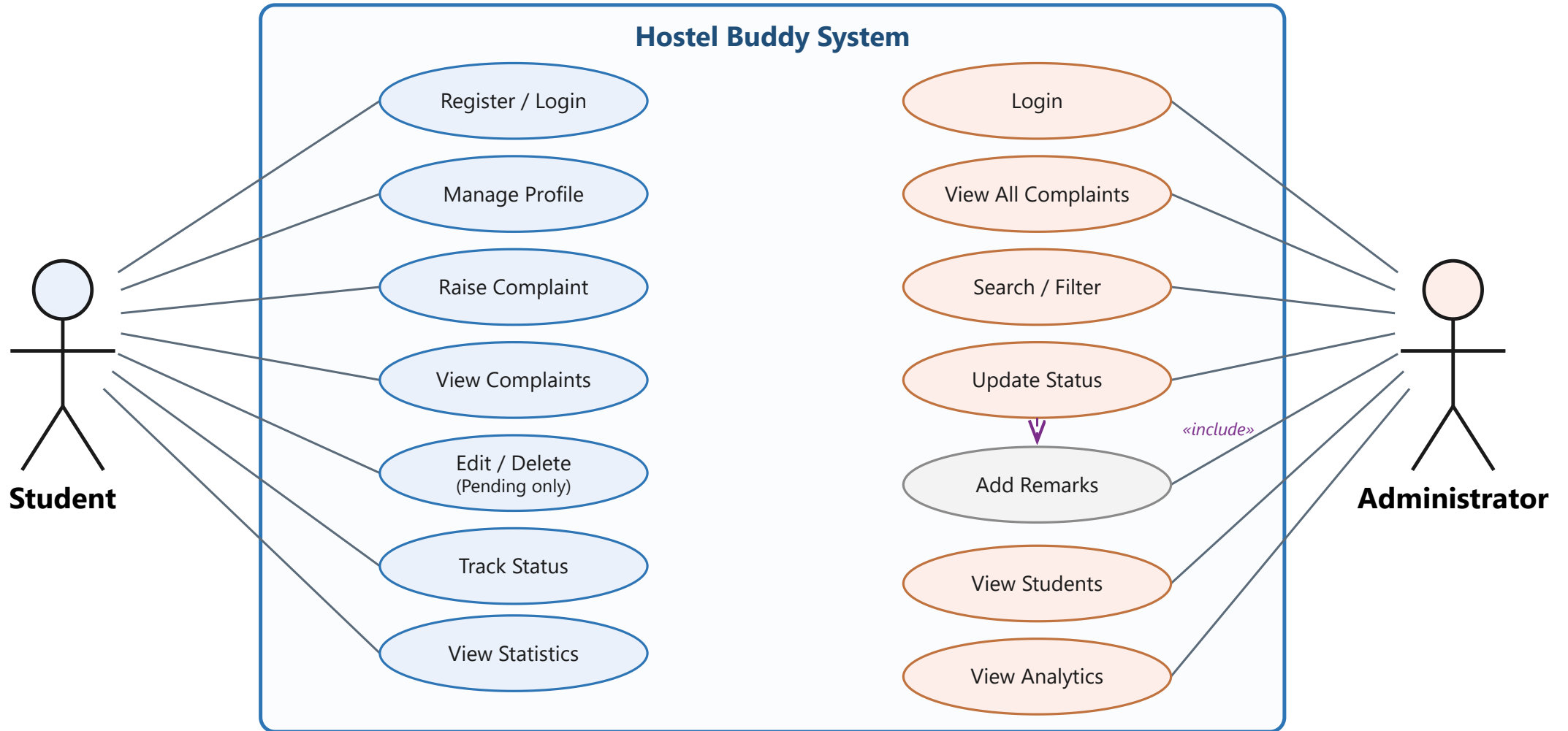
Non-Functional Requirements

- **Security** — hashed passwords, role-based access
- **Usability** — responsive, works on mobile
- **Maintainability** — independent modules
- **Reliability** — no complaint data loss

11 Use Case Diagram

Vijay

2 min (11)



Two actors only. Administrator drives every status transition; the student is read-only on status.

- 1 Hostel Buddy uses a **modular layered architecture** — four feature modules over a shared infrastructure layer.
- 2 **Business logic is separated** from presentation and database access — services hold the rules, repositories hold the SQL.
- 3 The **complaint lifecycle is controlled by authorization rules** — only the administrator advances status; students edit only while Pending.
- 4 The architecture is **maintainable and suitable for the current model version**, with a clear path to scale.

Thank you — questions?